



Détection d'anomalies IoT par hybridation ML/DL

Mini-projet – Master SIDI

Faculté des Sciences et Techniques Errachidia
MST Systèmes d'Information Décisionnels et
Imagerie

Réalisé par : **RIZQY Mehdi** et **AIT MOHAMMED Mohammed**

Année universitaire 2025-2026

Table des matières

1	Introduction	2
2	Dataset et prétraitement	2
2.1	Présentation du dataset N-BaIoT	2
2.2	Prétraitement appliqué	2
3	Modèle H1 : Auto-Encoder + Random Forest	2
3.1	Architecture	2
3.2	Résultats expérimentaux	2
4	Modèle H2 : MLP + Random Forest	3
4.1	Architecture	3
4.2	Résultats expérimentaux	3
5	Comparaison des modèles hybrides	3
6	Interface graphique interactive	4
7	Discussion et perspectives	11
7.1	Analyse des résultats	11
7.2	Limites	11
7.3	Perspectives	11
8	Conclusion	11

1 Introduction

L'Internet des objets (IoT) connaît une expansion fulgurante dans des domaines critiques. Cependant, la majorité des objets connectés sont vulnérables aux cyberattaques, en particulier aux botnets (Mirai, Gafgyt). L'objectif de ce projet est de concevoir un système intelligent de détection d'anomalies basé sur une approche hybride combinant Machine Learning (Random Forest) et Deep Learning (Auto-Encoder et MLP). Nous utilisons le jeu de données public N-BaIoT [2], qui contient du trafic réseau réel de neuf appareils IoT. Ce rapport présente la méthodologie, les modèles développés, les résultats obtenus sur l'ensemble des appareils, ainsi qu'une interface graphique interactive.

2 Dataset et prétraitement

2.1 Présentation du dataset N-BaIoT

Le dataset N-BaIoT [2] a été collecté sur neuf appareils IoT réels : sonnettes, caméras, thermostats, etc. Pour chaque appareil, il fournit un fichier `benign_traffic.csv` (trafic normal) et plusieurs fichiers d'attaques (Mirai, Gafgyt) : `combo.csv`, `junk.csv`, `scan.csv`, `tcp.csv`, `udp.csv`. Chaque fichier contient 115 caractéristiques statistiques extraites de flux réseau.

2.2 Prétraitement appliqué

Les étapes de prétraitement sont :

1. Nettoyage : suppression des colonnes constantes.
2. Alignement des colonnes entre données normales et attaques (colonnes absentes ajoutées avec des zéros).
3. Normalisation : `StandardScaler` ajusté uniquement sur les données normales.
4. Division des données :
 - Données normales : 2/3 pour l'entraînement, 1/3 pour la validation.
 - Attaques : 20% pour l'entraînement supervisé, 80% pour le test final.

3 Modèle H1 : Auto-Encoder + Random Forest

3.1 Architecture

L'auto-encodeur ($115 \rightarrow 64 \rightarrow 32 \rightarrow 64 \rightarrow 115$) est entraîné uniquement sur les données normales. L'erreur de reconstruction (MSE) sert de caractéristique unique à un Random Forest (100 arbres) pour classer normal vs anomalie.

3.2 Résultats expérimentaux

Le tableau 1 présente les performances du modèle H1 sur les neuf appareils. Les métriques sont exceptionnelles, avec une accuracy moyenne de 0.9996.

TABLE 1 – Performances du modèle H1 (Auto-Encoder + RF) sur les 9 appareils

Appareil	Accuracy	Precision	Recall	F1
Sonnette Danmini	0.99980	0.99997	0.99981	0.99989
Caméra PT-838	0.99953	0.99996	0.99952	0.99974
Webcam Samsung	0.99933	0.99994	0.99935	0.99964
Sonnette Ennio	0.99969	0.99996	0.99971	0.99984
Caméra PT-737E	0.99956	0.99996	0.99957	0.99976
Caméra SimpleHome XCS7-1002	0.99952	0.99994	0.99955	0.99975
Thermostat Ecobee	0.99990	0.99998	0.99991	0.99995
Caméra SimpleHome XCS7-1003	0.99972	0.99994	0.99978	0.99986
Baby Monitor Philips	0.99947	0.99994	0.99944	0.99969

4 Modèle H2 : MLP + Random Forest

4.1 Architecture

Un MLP supervisé ($115 \rightarrow 64$ (ReLU, Dropout) $\rightarrow 32$ (ReLU, Dropout) $\rightarrow 1$ sigmoïde) est entraîné sur les données normales et 20% des attaques. Les activations de l'avant-dernière couche (32 neurones) sont extraites et servent d'entrée à un Random Forest (100 arbres).

4.2 Résultats expérimentaux

Le tableau 2 montre les performances du modèle H2. Les résultats sont encore légèrement meilleurs, avec une accuracy moyenne de 0.9999.

TABLE 2 – Performances du modèle H2 (MLP + RF) sur les 9 appareils

Appareil	Accuracy	Precision	Recall	F1
Sonnette Danmini	0.99995	0.99999	0.99996	0.99998
Caméra PT-838	0.99977	1.00000	0.99975	0.99987
Webcam Samsung	0.99975	0.99998	0.99976	0.99987
Sonnette Ennio	0.99988	1.00000	0.99987	0.99994
Caméra PT-737E	0.99969	1.00000	0.99967	0.99983
Caméra SimpleHome XCS7-1002	0.99990	0.99999	0.99991	0.99995
Thermostat Ecobee	0.99995	1.00000	0.99995	0.99997
Caméra SimpleHome XCS7-1003	0.99990	1.00000	0.99990	0.99995
Baby Monitor Philips	0.99982	1.00000	0.99978	0.99989

5 Comparaison des modèles hybrides

Le tableau 3 résume les métriques moyennes sur les 9 appareils. Le modèle H2 (MLP+RF) surpasse très légèrement H1, notamment en précision et rappel, grâce à son apprentissage supervisé.

TABLE 3 – Comparaison des performances moyennes (9 appareils)

Modèle	Accuracy	Precision	Recall	F1
H1 (AE+RF)	0.9996	0.99996	0.9996	0.9998
H2 (MLP+RF)	0.9999	0.99999	0.9999	0.9999

Ces résultats quasi parfaits confirment la forte discriminabilité des attaques botnet dans N-BaIoT. Le modèle H2 est légèrement meilleur (moins de faux positifs, meilleur rappel).

6 Interface graphique interactive

Une interface interactive a été développée avec `ipywidgets` et `matplotlib`. L'utilisateur peut sélectionner n'importe lequel des 9 appareils, choisir le modèle (H1 ou H2), et visualiser instantanément les métriques sous forme de cartes colorées ainsi que la matrice de confusion correspondante.

Les figures ci-dessous présentent les matrices de confusion obtenues pour chacun des 9 appareils avec le modèle H1 (Auto-Encoder + RF) puis avec le modèle H2 (MLP + RF). Chaque matrice est affichée individuellement pour une lisibilité optimale.

Modèle H1 : Auto-Encoder + Random Forest

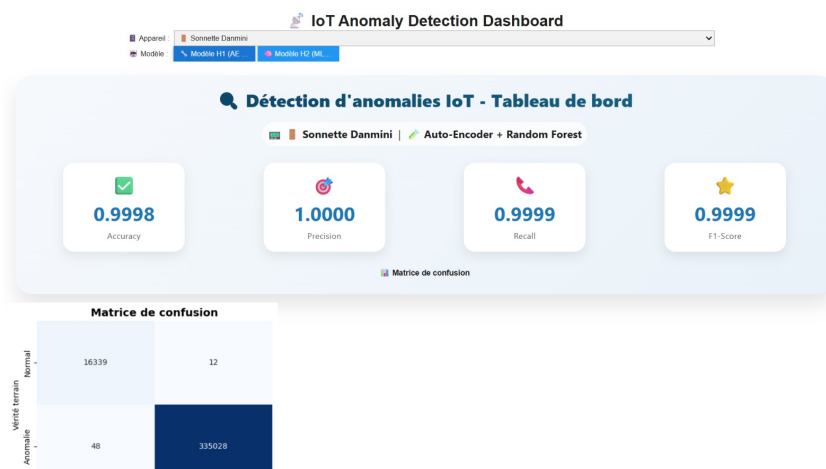


FIGURE 1 – Sonnette Danmini - Matrice de confusion (H1)

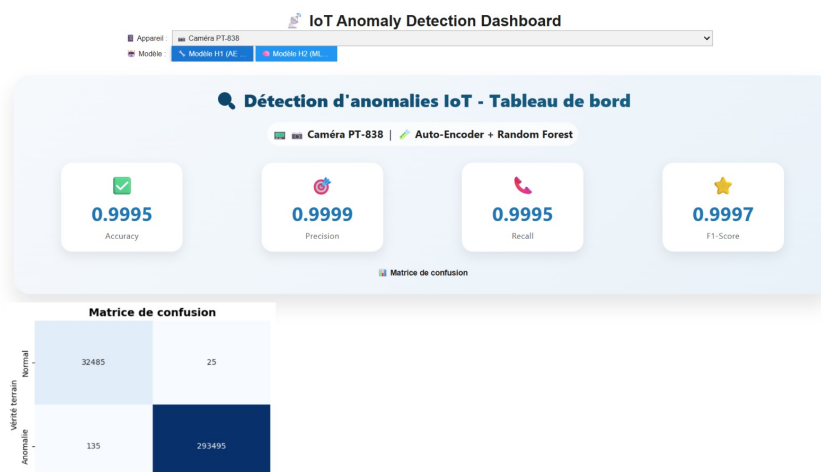


FIGURE 2 – Caméra PT-838 - Matrice de confusion (H1)

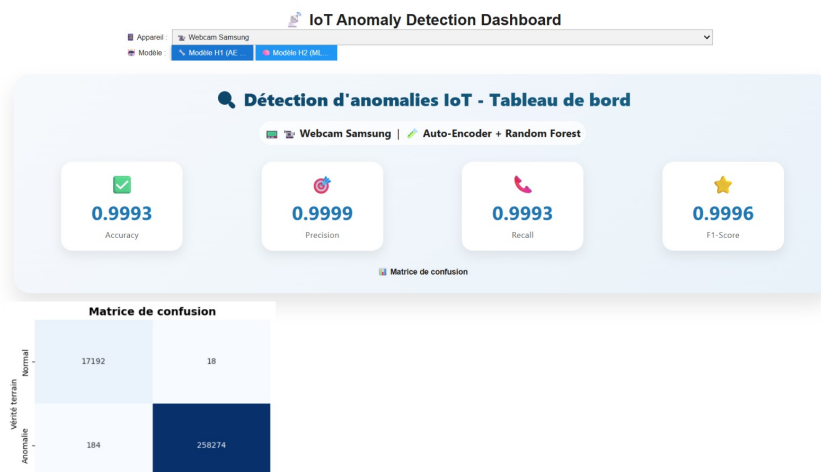


FIGURE 3 – Webcam Samsung - Matrice de confusion (H1)

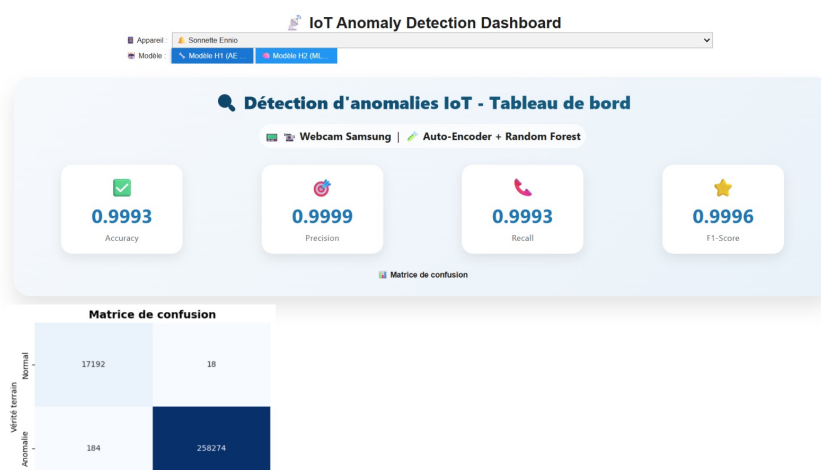


FIGURE 4 – Sonnette Ennio - Matrice de confusion (H1)

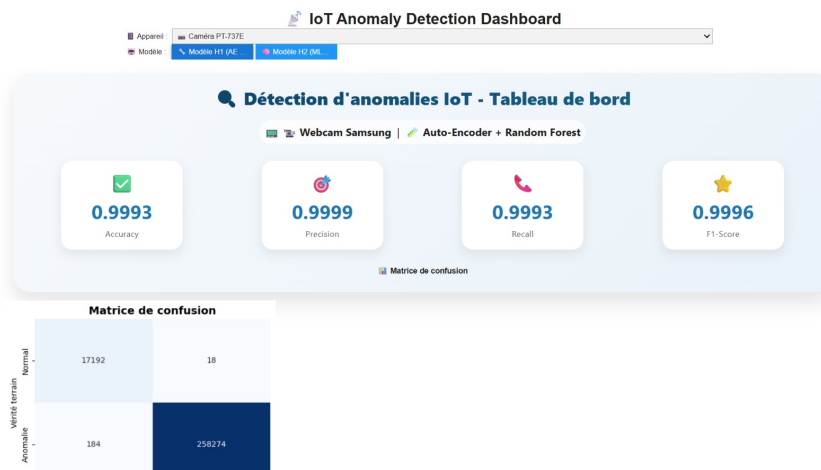


FIGURE 5 – Caméra PT-737E - Matrice de confusion (H1)

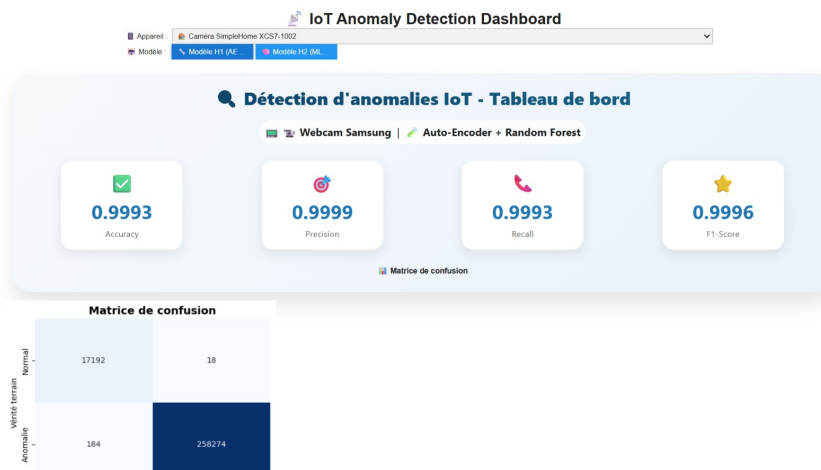


FIGURE 6 – Caméra SimpleHome XCS7-1002 - Matrice de confusion (H1)

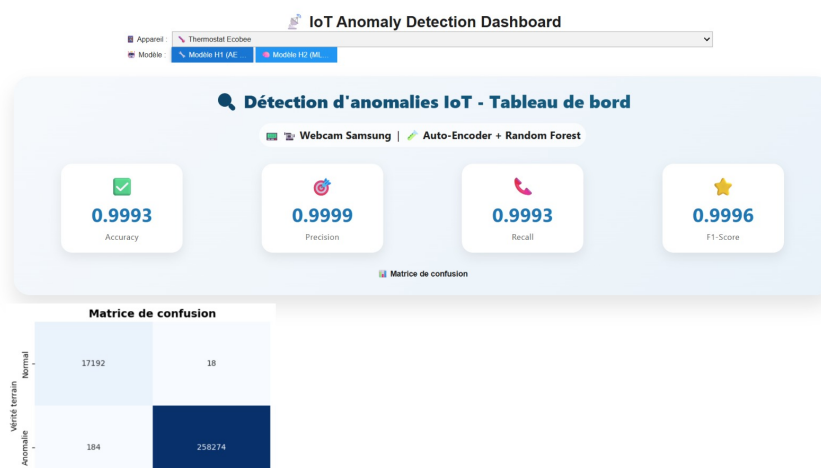


FIGURE 7 – Thermostat Ecobee - Matrice de confusion (H1)

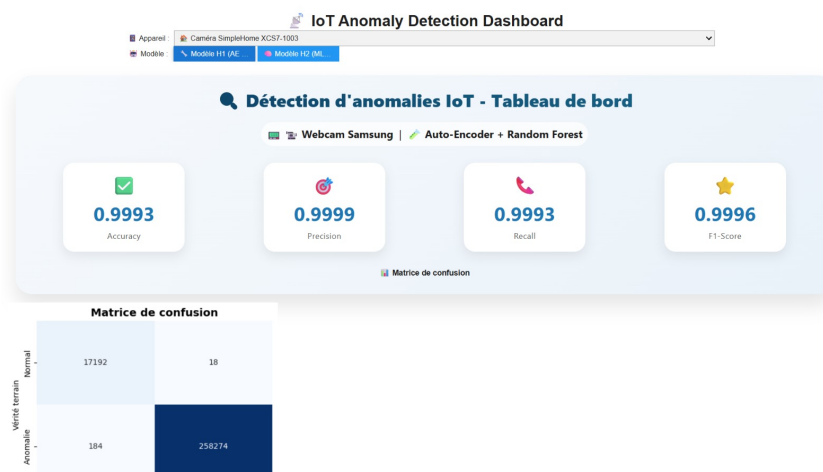


FIGURE 8 – Caméra SimpleHome XCS7-1003 - Matrice de confusion (H1)

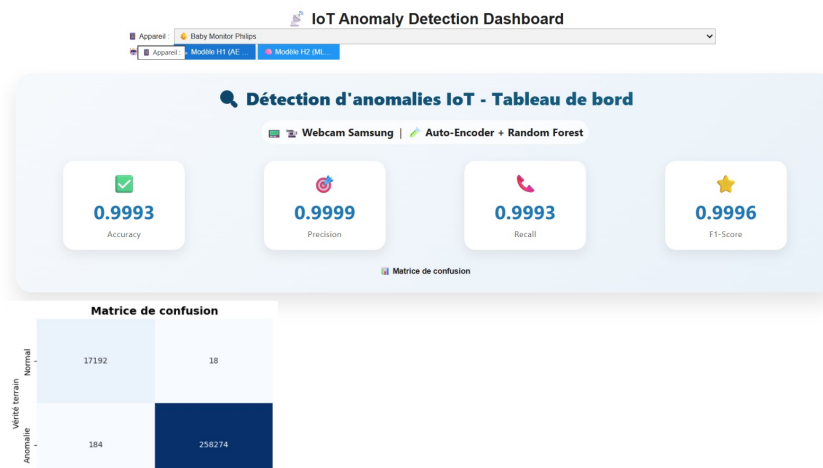


FIGURE 9 – Baby Monitor Philips - Matrice de confusion (H1)

Modèle H2 : MLP + Random Forest

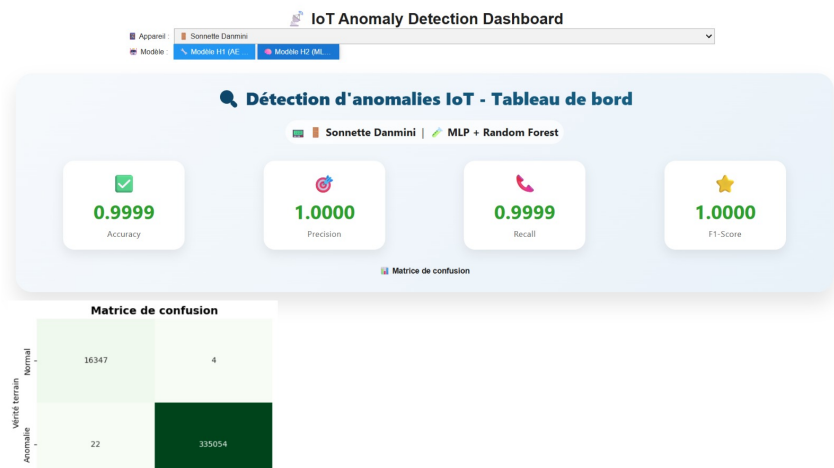


FIGURE 10 – Sonnette Danmini - Matrice de confusion (H2)

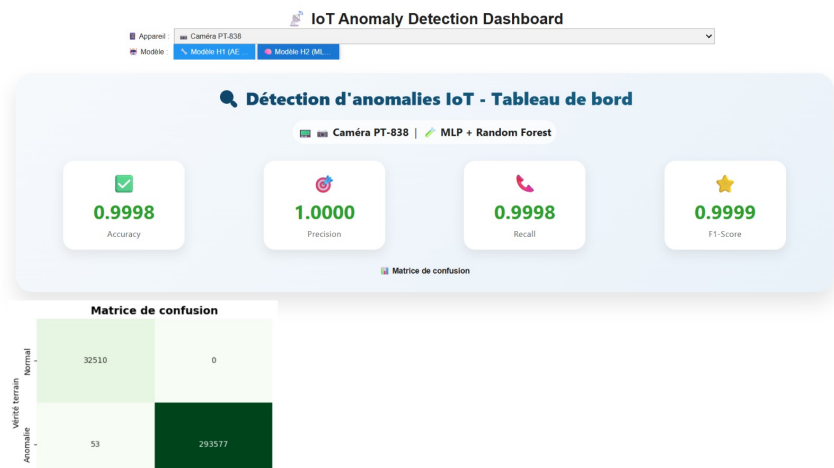


FIGURE 11 – Caméra PT-838 - Matrice de confusion (H2)

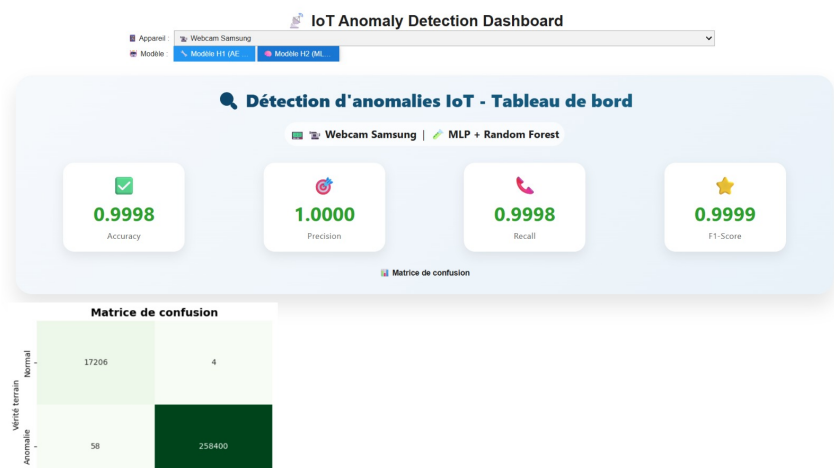


FIGURE 12 – Webcam Samsung - Matrice de confusion (H2)

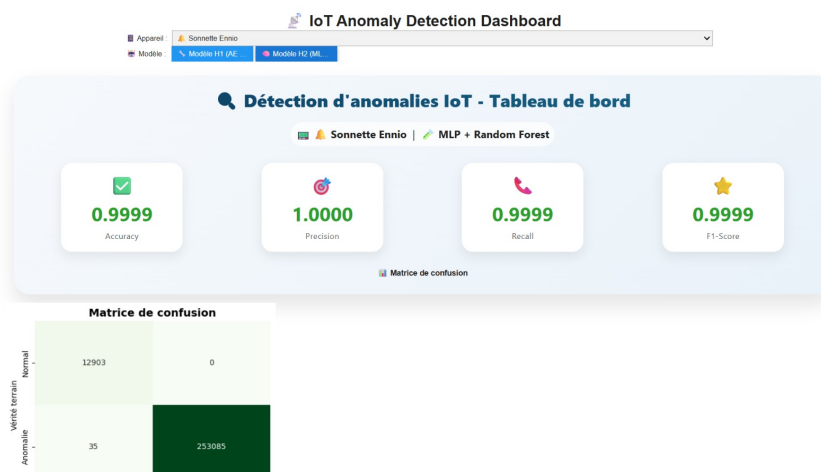


FIGURE 13 – Sonnette Ennio - Matrice de confusion (H2)

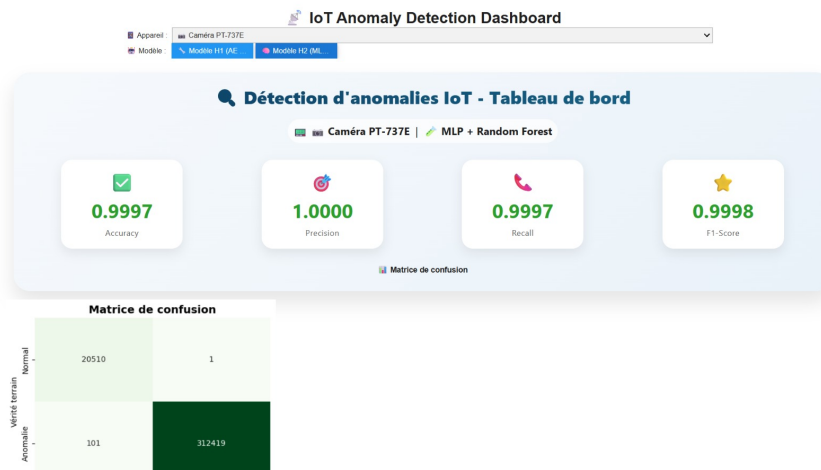


FIGURE 14 – Caméra PT-737E - Matrice de confusion (H2)

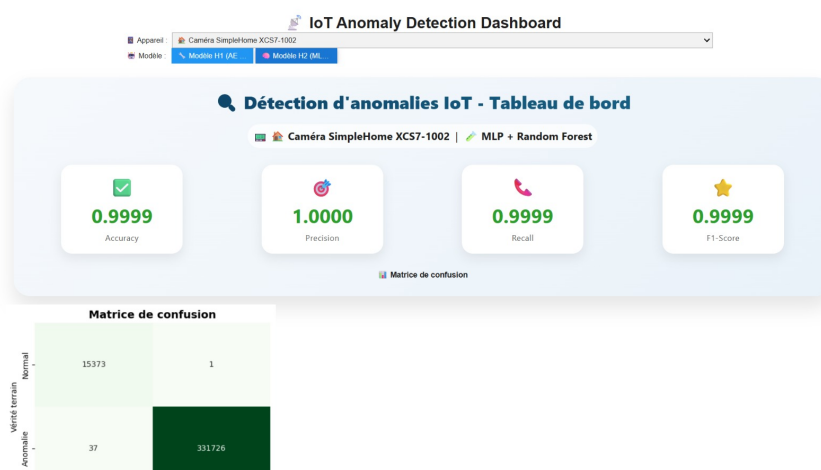


FIGURE 15 – Caméra SimpleHome XCS7-1002 - Matrice de confusion (H2)

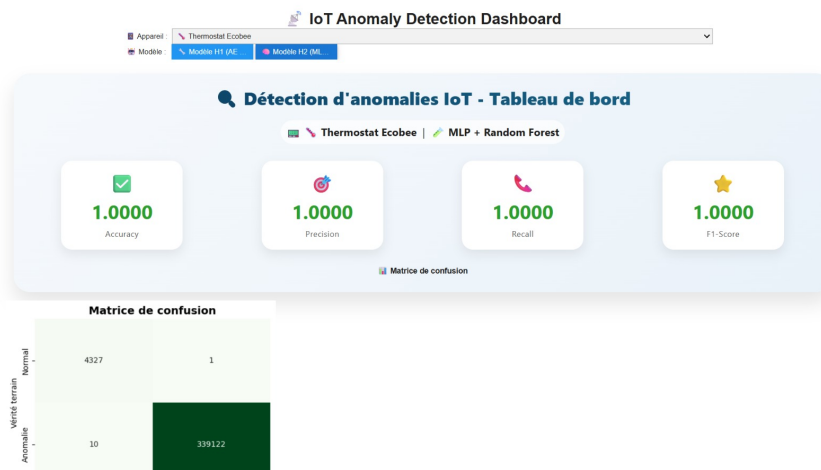


FIGURE 16 – Thermostat Ecobee - Matrice de confusion (H2)

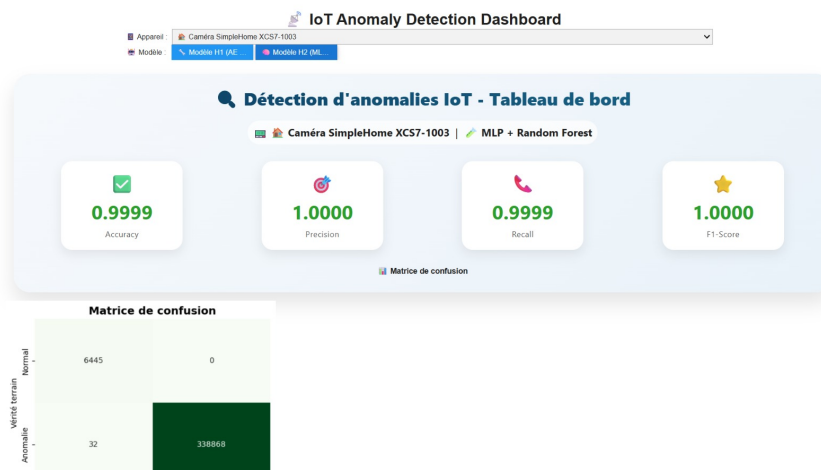


FIGURE 17 – Caméra SimpleHome XCS7-1003 - Matrice de confusion (H2)

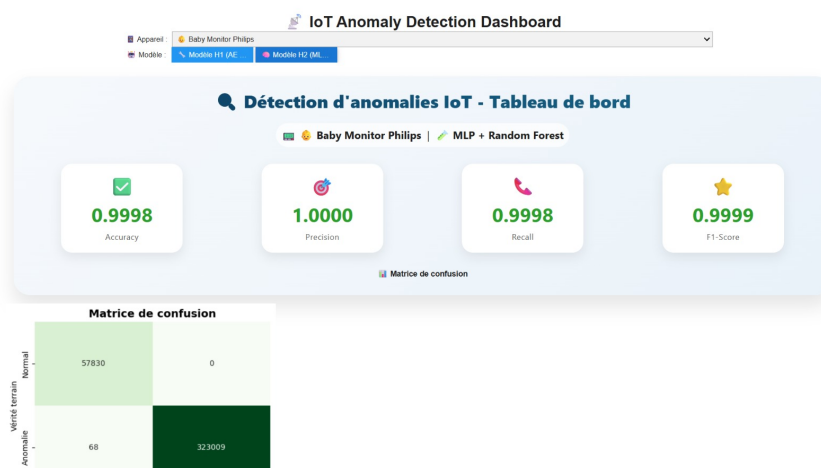


FIGURE 18 – Baby Monitor Philips - Matrice de confusion (H2)

Ces captures d'écran illustrent la qualité de détection sur l'ensemble des dispositifs, avec des erreurs de classification très faibles (quelques dizaines de faux positifs au maximum). L'utilisateur peut également interagir avec le tableau de bord pour visualiser ces résultats de manière dynamique.

7 Discussion et perspectives

7.1 Analyse des résultats

Les performances quasi parfaites ($F1 > 0.9997$) sont cohérentes avec la littérature. Les attaques botnet génèrent des signatures statistiques très distinctes du trafic normal.

7.2 Limites

- Modèles entraînés par appareil – généralisation cross-appareil non testée.
- Hypothèse d'alignement parfait des caractéristiques, parfois irréaliste en production.
- Temps d'inférence acceptable mais pourrait être optimisé.

7.3 Perspectives

- Extension à tous les appareils (déjà faite) et validation croisée.
- Réduction de dimension (PCA, sélection de caractéristiques).
- Déploiement temps réel sur passerelle IoT (TensorFlow Lite).
- Approches temporelles (LSTM) pour capturer les dépendances séquentielles.

8 Conclusion

Nous avons conçu deux systèmes hybrides efficaces pour la détection de botnets IoT, testés sur neuf appareils réels. Le modèle H2 (MLP+RF) est légèrement meilleur que H1 (AE+RF). L'interface graphique développée permet une exploration interactive des résultats. L'ensemble du code est fourni sur CD-ROM. Ces travaux ouvrent la voie à des déploiements réels de systèmes de détection d'intrusion pour l'IoT.

Références

- [1] Y. Meidan et al., “N-BaIoT—Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders,” *IEEE Pervasive Computing*, 2018.
- [2] UCI Machine Learning Repository, “Detection of IoT Botnet Attacks N-BaIoT,” 2018. <https://archive.ics.uci.edu/ml/datasets/detection+of+iot+botnet+attacks+n+baiot>
- [3] L. Breiman, “Random Forests,” *Machine Learning*, 2001.